

HIGH-PERFORMANCE & PARALLEL COMPUTING

Laboratory Report

Shared-Memory Concurrency & OpenMP Paradigms
Java Edition

Student ID 230103027

Platform Windows

CPU 11th Gen Intel Core i5-1155G7 @ 2.50 GHz

Java Java/Javac 21.0.1 (LTS)

Hardware: 4 physical cores, 8 logical processors.

```
PS C:\Users\kemel\Desktop\lab\OpenMP_Practice_230103027> Get-CimInstance Win32_Processor
logicalProcessors
```

Name	NumberOfCores	NumberOfLogicalProcessors
11th Gen Intel(R) Core(TM) i5-1155G7 @ 2.50GHz	4	8

Lab 1: Fork-Join Parallelism & Thread Oversubscription

Task 1.1 - Fork-Join Execution

The Java ForkJoinLab1 program was compiled and executed 10 consecutive times. The observed logical-rank print order was 2 -> 1 -> 3 -> 0 in all ten runs. The output was non-sequential, demonstrating that logical rank does not impose execution order.

Task 1.2 - Thread Oversubscription Sweep

P	Trial 1 (ms)	Trial 2 (ms)	Trial 3 (ms)	Mean (ms)
1	7.6326	7.1672	7.5760	7.4586
2	0.4808	0.4807	0.5209	0.4941
4	0.3657	0.3663	0.3448	0.3589
8	0.3511	0.3968	0.4080	0.3853
16	0.4679	0.5019	0.6325	0.5341
32	1.0538	0.6776	0.6738	0.8017
64	1.0821	0.9789	1.1683	1.0764

The first P=1 measurement is affected by JVM startup/class-initialization/JIT effects because it is always the first configuration in each process. From P=4/8 onward, increasing thread count eventually increases overhead. P>4 exceeds physical cores and P>8 exceeds available logical processors.

Task 1.3 - CPU Saturation

Threads	Work per thread	Execution time	Observed CPU utilization
8	500,000,000 square roots	1.4344 s	approximately 7% ->

		100% during computation
--	--	-------------------------

The workload briefly saturated the processor, with Task Manager showing CPU utilization rising to approximately 100% during the parallel computation.

Analytical Questions - Lab 1

Q1.1 Scheduling: Parallel threads execute concurrently, so logical rank does not determine print order.

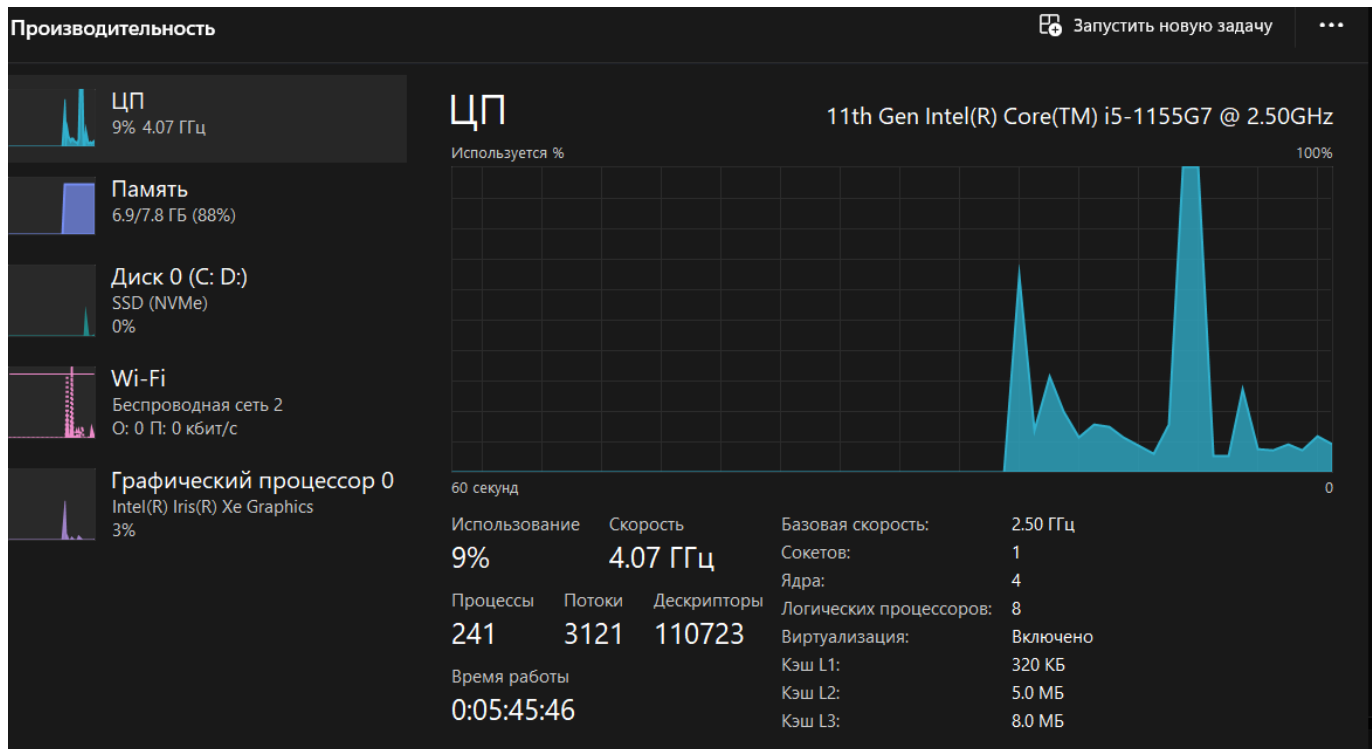
ForkJoinPool distributes work among JVM worker threads, while the operating-system scheduler determines when runnable threads execute on logical processors. In this experiment the ten runs happened to repeat 2 -> 1 -> 3 -> 0, but this ordering is not guaranteed by the program.

```
PS C:\Users\kemel\Desktop\lab\OpenMP_Practice_230103027> java ForkJoinLab1
Master thread starting. Spawning thread team of size: 4
[Worker] Logical Rank: 2 | Worker Thread: ForkJoinPool-1-worker-1 (OS ID: 21)
[Worker] Logical Rank: 1 | Worker Thread: ForkJoinPool-1-worker-2 (OS ID: 22)
[Worker] Logical Rank: 3 | Worker Thread: ForkJoinPool-1-worker-3 (OS ID: 23)
[Master] Logical Rank: 0 | Worker Thread: ForkJoinPool-1-worker-4 (OS ID: 24)
Parallel region closed. Execution returned to master thread.
```

Q1.2 Oversubscription: When software threads exceed available physical or logical execution resources, scheduling overhead grows. Context switches require thread state to be preserved/restored and competing threads can disturb cache locality. The measured sweep shows rising mean time at P=16, 32 and 64 compared with P=4/8.

```
PS C:\Users\kemel\Desktop\lab\OpenMP_
Thread Oversubscription Sweep
P,Time_ms
1,7.0897
2,0.5694
4,0.3573
8,0.3062
16,0.4567
32,0.6676
64,0.8984
```

Q1.3 Barriers: A barrier prevents execution from moving beyond a parallel region until required worker work has completed. In the Java implementation, join() provides the necessary waiting behavior. Without synchronization, subsequent code could observe incomplete shared results or inconsistent state.



Q1.4 Architectural Mapping: A hardware execution thread (SMT/Hyper-Threading) is a processor execution context; an OS kernel thread is scheduled by the operating system onto hardware contexts; a language/runtime thread is an abstraction managed through the language runtime. Java ForkJoinPool workers ultimately rely on OS scheduling and hardware execution resources.

Lab 2: Numerical Integration (Pi Approximation) & Parallel Reductions

Task 2.1 - Race Condition Quantification

```
PS C:\Users\kemel\Desktop\lab\OpenMP_Practice_230103027> java PiRaceCondition
Naive Race Condition - Pi Integration
N = 100000000

P=1 | Pi=3.141592653590 | Error=6.332712132462e-13 | Time=0.8629 s
P=2 | Pi=1.514425468173 | Error=1.627167185417e+00 | Time=0.9749 s
P=4 | Pi=0.979914652507 | Error=2.161678001083e+00 | Time=0.1930 s
P=8 | Pi=0.302229891375 | Error=2.839362762215e+00 | Time=0.1025 s
```

The naive shared accumulator is not thread-safe. Concurrent read-modify-write operations cause lost updates. With one thread the result is essentially correct; with multiple threads, the shared sum becomes increasingly corrupted in this observed run.

Task 2.2 - Critical Section Overhead

P	Calculated Pi	Absolute Error	Time (s)
1	3.141592653590	2.886579864025e-14	0.0253
2	3.141592653590	4.041211809636e-14	0.0188
4	3.141592653590	2.220446049250e-14	0.0061
8	3.141592653590	2.353672812205e-14	0.0066

Configuration: N = 1,000,000 steps, with synchronized(lock) applied to the shared update on every iteration. Correctness was maintained for all thread counts. The P=8 time was slightly worse than P=4 in this run, showing that additional threads do not necessarily improve a synchronization-heavy region.

Note: the manual asks for lock-contention overhead relative to a single-threaded baseline. The collected P=1 synchronized run is reported above; a separate unsynchronized serial baseline at N=1,000,000 was not measured, so a strict lock-overhead percentage is not fabricated in this report.

Task 2.3 - Strong Scaling Benchmark

P	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Average (s)
1	0.4711	0.4710	0.4743	0.4715	0.4655	0.47068
2	0.3125	0.3105	0.3086	0.3111	0.3122	0.31098
4	0.1961	0.1999	0.1966	0.1963	0.2003	0.19784
8	0.1405	0.1469	0.1432	0.1407	0.1533	0.14492
16	0.1893	0.1462	0.1381	0.1407	0.1388	0.15062

Task 2.4 - Speedup & Efficiency

P	Average T(P) (s)	Speedup S(P)	Efficiency E(P)
1	0.47068	1.000x	100.0%
2	0.31098	1.514x	75.7%
4	0.19784	2.379x	59.5%
8	0.14492	3.248x	40.6%
16	0.15062	3.125x	19.5%

Speedup was calculated as $S(P) = T(1) / T(P)$, and efficiency as $E(P) = S(P) / P$. The best measured average time occurred at P=8. At P=16, average time increased slightly, consistent with oversubscription on an 8-logical-processor system.